Deadlock Detector 아키텍처 문서

시스템 개요

Deadlock Detector는 운영체제의 데드락 감지 및 회복 알고리즘을 AI/LLM 다중 에이전트 시스템에 적용한 시스템입니다. Wait-For Graph 기반 사이클 탐지, 다양한 희생자 선택 전략, 은행원 알고리즘을 통한 데드락 회피를 제공합니다.

---

1. 시스템 아키텍처

1.1 전체 구조

```
graph LR
    subgraph "에이전트 계층"
        A1[Agent 1]
        A2[Agent 2]
        A3[Agent 3]
    end

    subgraph "자원 계층"
        R1[Resource 1]
        R2[Resource 2]
        R3[Resource 3]
    end

    subgraph "Wait-For Graph"
        WFG[Wait-For Graph<br/>Nodes + Edges]
    end

    subgraph "감지 계층"
        CD[Cycle Detector<br/>DFS Algorithm]
    end

    subgraph "회복 계층"
        VS[Victim Selector<br/>4 Strategies]
        RM[Rollback Manager<br/>Checkpoint Recovery]
    end

    subgraph "회피 계층"
        BA[Banker's Algorithm<br/>Safety Checker]
    end

    subgraph "데이터 계층"
        MongoDB[(MongoDB)]
        Redis[(Redis)]
    end

    A1 -.->|Waiting| R1
    A2 -.->|Holding| R1
    A2 -.->|Waiting| R2
    A3 -.->|Holding| R2
    A3 -.->|Waiting| R3
    A1 -.->|Holding| R3

    A1 --> WFG
    A2 --> WFG
```

```
    A3 --> WFG
    R1 --> WFG
    R2 --> WFG
    R3 --> WFG

    WFG --> CD
    CD --> VS
    VS --> RM

    CD --> BA

    WFG --> MongoDB
    WFG --> Redis

    style CD fill:#ef5350
    style VS fill:#ffa726
    style RM fill:#66bb6a
    style BA fill:#42a5f5
```

1.2 Wait-For Graph 구조

그래프 구성: - 노드 **(Nodes)**: 에이전트와 자원 - 엣지 **(Edges)**: 대기 관계 (Waiting) 또는 보유 관계 (Holding) - 사이클 **(Cycle)**: 데드락 상태를 나타내는 순환 경로

```
graph LR
    A1[Agent 1] -->|Waiting for| R1[Resource 1]
    A2[Agent 2] -->|Holding| R1
    A2 -->|Waiting for| R2[Resource 2]
    A3[Agent 3] -->|Holding| R2
    A3 -->|Waiting for| R3[Resource 3]
    A1 -->|Holding| R3

    style A1 fill:#ef5350
    style A2 fill:#ef5350
    style A3 fill:#ef5350
```

---

2. 데드락 감지 알고리즘

2.1 사이클 탐지 (Cycle Detection)

```
graph LR
    Start[Start Detection] --> Build[Build Wait-For Graph]
    Build --> DFS[DFS Traversal]
    DFS --> Visit{Visit Node}
    Visit -->|Not Visited| Mark[Mark as Visiting]
    Mark --> Recurse[Recurse Neighbors]
    Recurse --> Visit

    Visit -->|Already Visiting| Cycle[Back Edge Found]
    Visit -->|Fully Visited| Done[Mark as Visited]

    Cycle --> Found[Deadlock Detected]
    Found --> Extract[Extract Cycle Path]
    Extract --> Return[Return Cycles]

    Done --> More{More Nodes?}
    More -->|Yes| Visit
```

```
    More -->|No| NoCycle[No Deadlock]
    NoCycle --> Return

    style Cycle fill:#ef5350
    style Found fill:#ef5350
```

2.2 DFS 기반 사이클 탐지 구현

```
class CycleDetector {
  detect(graph: WaitForGraph): Cycle[] {
    const cycles: Cycle[] = [];
    const visited = new Set<string>();
    const visiting = new Set<string>();
    const path: string[] = [];

    const dfs = (nodeId: string): boolean => {
      if (visiting.has(nodeId)) {
        // 사이클 발견
        const cycleStart = path.indexOf(nodeId);
        const cyclePath = path.slice(cycleStart);
        cycles.push(this.extractCycle(graph, cyclePath));
        return true;
      }

      if (visited.has(nodeId)) {
        return false;
      }

      visiting.add(nodeId);
      path.push(nodeId);

      const neighbors = graph.getNeighbors(nodeId);
      for (const neighbor of neighbors) {
        if (dfs(neighbor.id)) {
          return true;
        }
      }

      path.pop();
      visiting.delete(nodeId);
      visited.add(nodeId);

      return false;
    };

    for (const node of graph.getNodes()) {
      if (!visited.has(node.id)) {
        dfs(node.id);
      }
    }

    return cycles;
  }

  private extractCycle(graph: WaitForGraph, path: string[]): Cycle {
    const agents: string[] = [];
    const resources: string[] = [];
```

```
    for (let i = 0; i < path.length; i++) {
      const node = graph.getNode(path[i]);
      if (node.type === 'agent') {
        agents.push(node.id);
      } else if (node.type === 'resource') {
        resources.push(node.id);
      }
    }

    return {
      cycleId: this.generateCycleId(agents, resources),
      agents,
      resources,
      detectedAt: new Date()
    };
  }
}
```

2.3 시간 복잡도

- 최악의 경우: O(V + E) - V는 노드 수, E는 엣지 수
- 평균 경우: O(V + E) - DFS의 효율적 구현
- 최적의 경우: O(V) - 사이클이 없는 경우

---

3. 희생자 선택 전략 (Victim Selection)

3.1 전략 비교

```
graph LR
    subgraph "우선순위 기반"
        P1[낮은 우선순위 선택]
    end

    subgraph "나이 기반"
        P2[젊은 에이전트 선택]
    end

    subgraph "자원 보유량 기반"
        P3[많은 자원 보유 선택]
    end

    subgraph "의존성 최소화"
        P4[적은 의존성 선택]
    end
```

3.2 전략별 구현

3.2.1 우선순위 기반 (Lowest Priority)

가장 낮은 우선순위를 가진 에이전트를 선택합니다.

```
class LowestPrioritySelector {
  select(cycle: Cycle, graph: WaitForGraph): Agent {
    const agents = cycle.agents.map(id => graph.getAgent(id));

    // 우선순위 기준 정렬
    agents.sort((a, b) => a.priority - b.priority);
```

```
    // 가장 낮은 우선순위 반환
    return agents[0];
  }
}
```

장점: - 비즈니스 로직에 맞는 선택 - 중요한 에이전트 보호 - 구현이 간단함

단점: - 낮은 우선순위 에이전트의 기아 현상

3.2.2 나이 기반 (Youngest)

가장 최근에 생성된 에이전트를 선택합니다.

```
class YoungestSelector {
  select(cycle: Cycle, graph: WaitForGraph): Agent {
    const agents = cycle.agents.map(id => graph.getAgent(id));

    // 생성 시간 기준 정렬
    agents.sort((a, b) =>
      a.createdAt.getTime() - b.createdAt.getTime()
    );

    // 가장 최신 에이전트 반환
    return agents[0];
  }
}
```

장점: - 진행 중인 작업이 최소인 에이전트 선택 - 롤백 비용이 낮음

단점: - 새로운 에이전트의 불리함

3.2.3 자원 보유량 기반 (Most Resources)

가장 많은 자원을 보유한 에이전트를 선택합니다.

```
class MostResourcesSelector {
  select(cycle: Cycle, graph: WaitForGraph): Agent {
    const agents = cycle.agents.map(id => graph.getAgent(id));

    // 보유 자원 수 기준 정렬
    agents.sort((a, b) => b.heldResources.length - a.heldResources.length);

    // 가장 많은 자원을 보유한 에이전트 반환
    return agents[0];
  }
}
```

장점: - 자원 해제 효과가 큼 - 많은 에이전트가 혜택

단점: - 중요한 에이전트가 희생될 수 있음

3.2.4 의존성 최소화 (Least Dependencies)

가장 적은 수의 다른 에이전트가 의존하는 에이전트를 선택합니다.

```
class LeastDependenciesSelector {
  select(cycle: Cycle, graph: WaitForGraph): Agent {
    const agents = cycle.agents.map(id => graph.getAgent(id));

    // 의존성 수 기준 정렬
    agents.sort((a, b) => {
```

```
      const aDeps = graph.countDependents(a.id);
      const bDeps = graph.countDependents(b.id);
      return aDeps - bDeps;
    });

    // 가장 적은 의존성을 가진 에이전트 반환
    return agents[0];
  }
}
```

장점: - 부작용(side effect) 최소화 - 시스템 안정성 유지

단점: - 의존성 계산의 복잡도

---

4. 은행원 알고리즘 (Banker's Algorithm)

4.1 안전 상태 검사

```
graph LR
    Start[Request Resources] --> Check1{Request ≤ Available?}
    Check1 -->|No| Wait[Wait]
    Check1 -->|Yes| Simulate[Simulate Allocation]
    Simulate --> Check2{Safe State?}
    Check2 -->|Yes| Allocate[Allocate Resources]
    Check2 -->|No| Wait

    Wait --> Available[Available Resources]
    Available --> Available2[Available + Released]
    Available2 --> Check1

    style Wait fill:#ffa726
    style Check2 fill:#ef5350
    style Allocate fill:#66bb6a
```

4.2 안전 상태 검사 구현

```
class SafetyChecker {
  isSafe(
    allocation: Matrix,
    max: Matrix,
    available: number[]
  ): { isSafe: boolean; safeSequence: string[] } {
    const work = [...available];
    const finish = new Map<string, boolean>();

    for (const agentId of allocation.keys()) {
      finish.set(agentId, false);
    }

    const safeSequence: string[] = [];

    while (safeSequence.length < finish.size) {
      let found = false;

      for (const [agentId, _] of finish) {
        if (finish.get(agentId)) continue;

        const need = this.calculateNeed(allocation, max, agentId);
```

```
        const canFinish = this.leq(need, work);

        if (canFinish) {
          // 에이전트가 완료될 수 있음
          work = this.add(work, allocation.get(agentId));
          finish.set(agentId, true);
          safeSequence.push(agentId);
          found = true;
          break;
        }
      }

      if (!found) {
        // 안전 상태가 아님
        return { isSafe: false, safeSequence: [] };
      }
    }

    return { isSafe: true, safeSequence };
  }

  private calculateNeed(
    allocation: Matrix,
    max: Matrix,
    agentId: string
  ): number[] {
    const maxArr = max.get(agentId);
    const allocArr = allocation.get(agentId);
    return maxArr.map((m, i) => m - allocArr[i]);
  }

  private leq(a: number[], b: number[]): boolean {
    return a.every((val, i) => val <= b[i]);
  }

  private add(a: number[], b: number[]): number[] {
    return a.map((val, i) => val + b[i]);
  }
}
```

---

5. 체크포인트 및 롤백 (Checkpoint & Rollback)

5.1 체크포인트 프로세스

```
sequenceDiagram
    participant A as Agent
    participant CM as Checkpoint Manager
    participant DB as Database

    A->>CM: Create Checkpoint
    CM->>CM: Serialize State
    CM->>CM: Calculate Hash
    CM->>DB: Store Checkpoint
    DB-->>CM: Confirmation
    CM-->>A: Checkpoint ID
```

```
    Note over A,DB: Agent continues execution

    A->>CM: Rollback Request
    CM->>DB: Retrieve Checkpoint
    DB-->>CM: Checkpoint Data
    CM->>CM: Verify Integrity
    CM->>CM: Deserialize State
    CM-->>A: Restored State
```

5.2 상태 직렬화

```
class StateSerializer {
  serialize(state: any): Buffer {
    try {
      const json = JSON.stringify(state);
      return Buffer.from(json, 'utf-8');
    } catch (error) {
      throw new SerializationError('Failed to serialize state', error);
    }
  }

  deserialize(buffer: Buffer): any {
    try {
      const json = buffer.toString('utf-8');
      return JSON.parse(json);
    } catch (error) {
      throw new DeserializationError('Failed to deserialize state', error);
    }
  }

  calculateHash(buffer: Buffer): string {
    return crypto.createHash('sha256').update(buffer).digest('hex');
  }
}
```

---

6. 데이터 흐름

6.1 데드락 감지 흐름

```
sequenceDiagram
    participant M as Main System
    participant WFG as Wait-For Graph
    participant CD as Cycle Detector
    participant VS as Victim Selector
    participant RM as Rollback Manager
    participant DB as Database

    M->>WFG: Update Graph (Resource Request)
    WFG->>DB: Persist Graph

    M->>CD: Detect Deadlock
    CD->>WFG: Get Graph
    CD->>CD: Run DFS
    CD-->>M: Cycles Found

    alt Deadlock Detected
        M->>VS: Select Victim
```

```
        VS->>WFG: Analyze Cycle
        VS-->>M: Victim Agent

        M->>RM: Create Checkpoint for Victim
        RM->>DB: Store State

        M->>RM: Rollback Other Agents
        RM->>DB: Retrieve Checkpoints
        RM-->>M: Restored States

        M->>M: Release Resources
    end
```

6.2 자원 요청 흐름

```
sequenceDiagram
    participant A as Agent
    participant API as API Controller
    participant BA as Banker's Algorithm
    participant WFG as Wait-For Graph
    participant R as Resource Manager

    A->>API: Request Resource
    API->>BA: Check Safety
    BA->>BA: Simulate Allocation
    BA-->>API: Safe/Unsafe

    alt Safe State
        API->>R: Allocate Resource
        R-->>API: Allocated
        API->>WFG: Update Graph (Holding)
        API-->>A: Success
    else Unsafe State
        API->>WFG: Update Graph (Waiting)
        API-->>A: Wait (Blocked)
    end
```

---

7. 도메인 모델

7.1 핵심 도메인

```typescript
// 에이전트
interface Agent {
  id: string;
  name: string;
  priority: number;           // 1-10, 높을수록 우선
  status: 'active' | 'waiting' | 'terminated';
  heldResources: string[];   // 보유 중인 자원 ID
  waitingFor: string | null; // 대기 중인 자원 ID
  createdAt: Date;
  metadata?: Record<string, any>;
}

// 자원
interface Resource {
  id: string;
  name: string;
```

```
  type: 'computational' | 'storage' | 'network';
  instances: number;          // 총 인스턴스 수
  available: number;          // 사용 가능한 인스턴스
  allocatedTo: string[];      // 할당된 에이전트 ID
  createdAt: Date;
}

// 대기 엣지
interface WaitForEdge {
  from: string;               // 에이전트 ID
  to: string;                 // 자원 ID
  type: 'waiting' | 'holding';
  createdAt: Date;
}

// 사이클 (데드락)
interface Cycle {
  cycleId: string;
  agents: string[];           // 사이클에 포함된 에이전트
  resources: string[];        // 사이클에 포함된 자원
  detectedAt: Date;
}

// 희생자 선택 결과
interface VictimSelection {
  victim: Agent;
  strategy: 'lowest_priority' | 'youngest' | 'most_resources' | 'least_dependencies';
  reason: string;
  selectedAt: Date;
}
```

---

8. 성능 최적화

8.1 그래프 최적화

```
class OptimizedWaitForGraph {
  private adjacencyList: Map<string, Set<string>> = new Map();
  private nodeCache: Map<string, Node> = new Map();

  addEdge(from: string, to: string): void {
    if (!this.adjacencyList.has(from)) {
      this.adjacencyList.set(from, new Set());
    }
    this.adjacencyList.get(from)!.add(to);
  }

  getNeighbors(nodeId: string): Node[] {
    const neighbors = this.adjacencyList.get(nodeId) || new Set();
    return Array.from(neighbors)
      .map(id => this.nodeCache.get(id))
      .filter(Boolean) as Node[];
  }
}
```

8.2 캐싱 전략

```
class GraphCache {
  private cache: Map<string, { graph: WaitForGraph; timestamp: number }> = new Map();
  private ttl: number = 5000; // 5초

  get(agentId: string): WaitForGraph | null {
    const cached = this.cache.get(agentId);
    if (!cached) return null;

    if (Date.now() - cached.timestamp > this.ttl) {
      this.cache.delete(agentId);
      return null;
    }

    return cached.graph;
  }

  set(agentId: string, graph: WaitForGraph): void {
    this.cache.set(agentId, {
      graph,
      timestamp: Date.now()
    });
  }
}
```

---

9. 기술 스택

계층	기술	버전	용도
언어	TypeScript	5.9	타입 안전성
런타임	Node.js	20 LTS	서버 런타임
웹 프레임워크	Express.js	4.18	REST API
실시간 통신	Socket.IO	4.7	WebSocket
데이터베이스	MongoDB	7.0	영구 저장
캐싱	Redis	7.2	그래프 캐싱
테스트	Vitest	Latest	단위/통합 테스트

---

10. OS 개념 매핑 상세

OS 개념	적용	구현
Deadlock	Resource Competition	다중 에이전트 자원 경합
Wait-For Graph	Dependency Graph	에이전트-자원 대기 그래프
Cycle Detection	DFS	O(V + E) 사이클 탐지
Victim Selection	Process Termination	4가지 선택 전략
Banker's Algorithm	Deadlock Avoidance	안전 상태 기반 자원 할당
Checkpoint/Restore	State Recovery	에이전트 상태 저장/복구

문서 버전: 1.0.0 최종 업데이트: 2025-01-25 유지보수 담당자: Deadlock Detector 팀